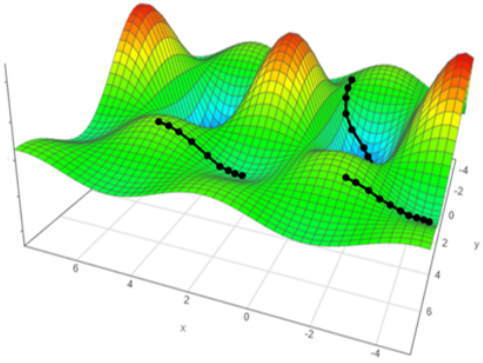
Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

Initialization and Normalization

Why Initialization Matters

- DL architectures are complex and leading to highly complex and nonconvex loss landscapes
- Initialization tends to determine the quality of of the solution found by iterative methods
- Initialization tends to impact the gradient information used by SGD throughout training



- Consider an MLP

$$h_j = \sigma(W_j h_{j-1} + b_j) \in \mathbb{R}^{d_j}, \quad , j = 1, \ldots, L, \quad , h_0 = x \in \mathbb{R}^d, \quad , h_L = y \in \mathbb{R} \tag{1}$$

where $b_j \in \mathbb{R}^{d_j}$ and $W_j \in \mathbb{R}^{d_j \times d_{j-1}}$ are learnable parameters of layer j.

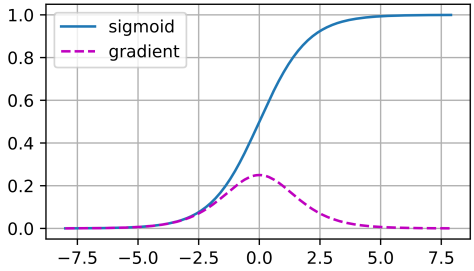
- Gradient is our learning signal and used in SGD for training.
- By Chain rule and treating each W_j as a $d_j^{j-1} := d_j \times d_{j-1}$ vector,

$$\begin{aligned} \partial_{W_j} \text{loss} &= \partial_{W_j} h_L \times \partial_{h_L} \text{loss} = \partial_{W_j} h_j \cdot \partial_{h_j} h_{j+1} \ldots \partial_{h_{L-2}} h_L \cdot \partial_{h_{L-1}} h_L \times \partial_{h_L} \text{loss} \\ &= \partial_{W_j} h_j \times M_{j+1} \times \ldots M_{L-1} \times M_L = \partial_{W_j} h_j \times \prod_{k=j+1}^{L} M_k \times \partial_{h_L} \text{loss} \end{aligned} \tag{2}$$

Side note: that the matrix $\partial_{W_j} h_L$ has dimension $d_j^{j-1} \times d_L$, the matrix $\partial_{W_j} h_j$ has dimension $d_j^{j-1} \times d_j$, while matrices M_k have dimensions $d_{k-1} \times d_k$. thus, dimension of both sides match.

Vanishing/Exploding Gradient Problems

- Our learning signal is proportional to $\prod_{k=j+1}^{L} M_k$, a product that could be very large or very small depending on the these matrices (and notably their eigen values).
- Vanishing gradient: Very small product means we cannot move very far from the initialization (happening with deep networks and RNNs; fixes include LSTM, skip connection).
- Exploding gradient: Very large product means divergence and instability (happening with deep networks; fixes include gradient clipping)
- Activation choice also plays a role



Another Problem: Symmetry and Limited Capacity



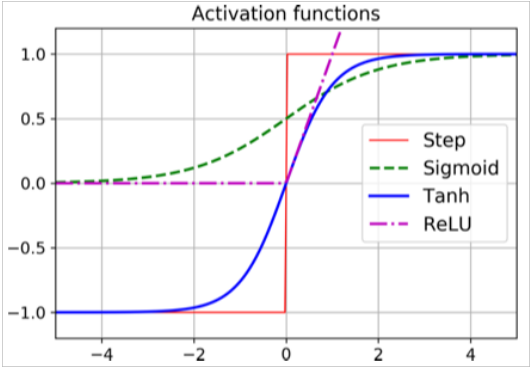
- Recall the motivation behind introducing nonlinearity and deep models was increased learning capacity
- With a bad initialization, we cannot achieve this. Consider

$$\begin{bmatrix} h(1) \\ h(2) \end{bmatrix} = \sigma\left(\begin{bmatrix} W_{11}x(1) + W_{12}x(2) + b(1) \\ W_{21}x(1) + W_{22}x(2) + b(2) \end{bmatrix} \right) \tag{3}$$

- Assume our initialization is such that $W_{11} = W_{12} = W_{21} = W_{22} = \alpha$ and $b(1) = b(2) = \beta$. thus, $h(1) = h(2)$
- then, the gradients $\partial_{W_{ij}} h(1)$ and $\partial_{W_{ij}} h(2)$ are equal. Thus, using gradient-based updates, W_{ij}'s will be the same, and so will the outputs
- Essentially, due to a symmetry, our two neurons reduce to only one neuron, hence limited capacity
- Randomization could help to solve this and the vanishing/exploding gradient at the same time.

Xavier Initialization

- Idea: Weights initialized such that the signal strength moving from one side to the other side in the network is preserved
- Thus, no drastic reduction or amplification leading to vanishing and exploding gradient, respectively
- Let us consider linear activation of the linear regime of all activations (small signal regime)



Xavier Initialization (Cont'd)

- Consider a layer with $W \in \mathbb{R}^{d_{out} \times d_{in}}$ and call the input and outputs x and o for simplicity.
- Forward information propagation condition

$$\mathbb{E}\|o\|^2 = \sum_{i=1}^{d_{out}} \mathbb{E}[o_i^2] = \sum_{i=1}^{d_{out}} \mathbb{E}[\sum_{j=1}^{d_{in}} W_{ij} x_j]^2 \tag{4}$$

- Backward information propagation condition

$$\mathbb{E}\|x\|^2 = \sum_{j=1}^{d_{in}} \mathbb{E}[x_j^2] = \sum_{j=1}^{d_{in}} \mathbb{E}[\sum_{i=1}^{d_{out}} W_{ij} o_i]^2 \tag{5}$$

- We assume $\mathbb{E}[x_j^2] = \gamma_{in}^2$, $\mathbb{E}[o_i^2] = \gamma_{out}^2$, $\mathbb{E}[W_{ij}^2] = \sigma^2$, and that W_{ij}'s are zero mean
- Thus, our conditions imply

$$d_{out}\gamma_{out}^2 = d_{out} d_{in} \sigma^2 \gamma_{in}^2, \qquad d_{in}\gamma_{in}^2 = d_{in} d_{out} \sigma^2 \gamma_{out}^2 \tag{6}$$

Xavier Initialization (Cont'd)



- Imposing $\gamma_{in}^2 = \gamma_{out}^2$

$$\begin{aligned} \gamma_{out}^2 &= d_{in}\sigma^2\gamma_{in}^2, \qquad \gamma_{in}^2 = d_{out}\sigma^2\gamma_{out}^2 \\ 1 &= d_{in}\sigma^2, \qquad 1 = d_{out}\sigma^2 \end{aligned} \tag{7}$$

- Thus, $\sigma^2 = 1/d_{in}$ and $\sigma^2 = 1/d_{out}$. As $d_{in} \neq d_{out}$, we instead set

$$\sigma^2 = \frac{1}{\mathrm{avg}(d_{in}, d_{out})} \qquad W_{ij} \sim \mathcal{N}(0, \sigma^2), \qquad \text{or} \quad W_{ij} \sim \mathcal{U}(-\sigma, \sigma) \tag{8}$$

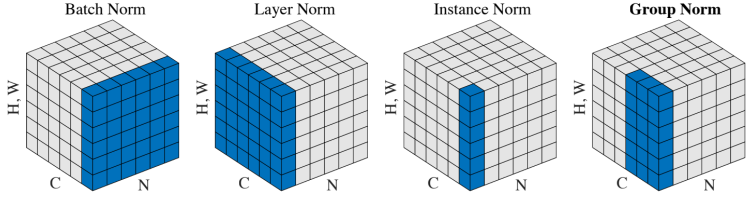
- Averaging can be arithmetic $(d_{in} + d_{out})/2$ or geometric mean $\sqrt{d_{in} \times d_{out}}$
- For ReLU's we typically increase σ^2 by a factor of 2 to compensate for the fact that half of the signal is dropped

$$\mathbb{E}[x_j^2] = \gamma_{in}^2 \quad \Leftrightarrow \quad \mathbb{E}[\max(0, x_j^2)] = \gamma_{in}^2/2 \tag{9}$$

for a symmetric input x.

Normalization Methods

- Xavier Initialization essentially performs a normalization at initialization
- As the weights get updated, weights will change, and so will the strength of layer's output/activation
- Normalization methods extend this idea to normalize the outputs throughout training.
- Certain connection to adaptive training methods to ensure parameters are varying more or less with the same rate
- Normalizing activations could lead to "smaller gradients", hence lower effective Lipschitzness and smoothness parameters G and L, thereby accelerating convergence.



Batch Normalization



- Consider a batch $\mathcal{B}$ of B vectors: $u_1, \ldots, u_B \in \mathbb{R}^p$ (p = $H \times W$ for a $H \times W$ matrix)
- Think of these as activations associated with samples/data used in minibatch SGD
- Batch normalization utilizes a data-driven operation

$$\mathrm{BN}(u) = \alpha \odot \frac{u - \mu_B}{\sigma_B + \kappa} + \beta, \quad \forall u \in \mathcal{B} \tag{10}$$

here $\alpha, \beta \in \mathbb{R}^p$ are learnable parameters and

$$\mu_B = \frac{1}{B} \sum_{u \in \mathcal{B}} u \in \mathbb{R}^p, \qquad \sigma_B^2 = \frac{1}{B} \sum_{u \in \mathcal{B}} \|u - \mu_B\|^2 \in \mathbb{R} \tag{11}$$

are sample mean and variance.

- $\kappa > 0$ introduced for numerical stability and avoiding division by zero.
- Moderate B to avoid loss of information (small batch) and slow computation (large batch)

Batch Normalization (Cont'd)



- Moderate B to avoid loss of information (small batch) and slow computation (large batch)
- Moderate B leads to a noisy calculation during the training leads to better generalization.
- How to apply in conjunction with other operations?

$$h = \text{Dropout}(\sigma(\text{BN}(Wx + b))) \tag{12}$$

- During inference, we use the statistics of the entire data to calculate the mean and variance of each layer's activation

$$\mu_{\mathcal{D}} = \frac{1}{n}\sum_{u \in \mathcal{D}} u \in \mathbb{R}^p, \qquad \sigma^2_{\mathcal{D}} = \frac{1}{n}\sum_{u \in \mathcal{D}} \|u - \mu_{\mathcal{D}}\|^2 \tag{13}$$

Which can be approximated on the fly from mini-batch statistics found during training:

$$\mu_{\mathcal{D}} \leftarrow \frac{1}{t}\mu_{B_t} + \frac{t}{t+1}\mu_{\mathcal{D}}, \qquad \sigma^2_{\mathcal{D}} \leftarrow \frac{1}{t}\sigma^2_{B_t} + \frac{t}{t+1}\sigma^2_{\mathcal{D}} \tag{14}$$

Layer Normalization

- A deterministic operation normalizing each vector u individually

$$\mathrm{LN}(u) = \frac{u - \mu_u 1}{\sigma_u + \kappa} \in \mathbb{R}^p, \quad \mu_u = \frac{1}{p}\sum_{i=1}^{p} u(i) \in \mathbb{R}, \quad \sigma_u = \frac{1}{p}\sum_{i=1}^{p} |u(i) - \mu_u|^2 \in \mathbb{R} \tag{15}$$

- Almost scale free $\mathrm{LN}(u) = \mathrm{LN}(\alpha u)$, a property useful for stabilizing training
- Can be applied to image-like quantities with multiple channels (e.g., $C = 3$)

